



Fundamentos de Aprendizaje Automático 2026/2027

PRÁCTICA Nº 1: Naive Bayes, K-NN y Scikit Learn

1. Objetivo

El objetivo de esta práctica es implementar los algoritmos de clasificación *Naive-Bayes* y *Vecinos Próximos* (K-NN). El algoritmo *Naive-Bayes* clasifica cada ejemplo en la clase cuya probabilidad a posteriori es máxima, suponiendo que todos los atributos con los que se representan las instancias son independientes. Por su parte, el algoritmo *K-NN* es un método que trata de clasificar cada ejemplo en función de la clase mayoritaria de los vecinos a ese ejemplo, considerando como vecino todo aquel que está a una determinada distancia, como puede ser la distancia Euclídea.

Tras nuestra implementación utilizaremos las librerías proporcionadas por Scikit Learn para Naive Bayes y K-NN, de manera que podamos comparar los resultados de ambas alternativas.

2. Conjuntos de datos

Para probar los algoritmos Naive Bayes y Vecinos Próximos utilizaremos los conjuntos de datos *fuga_telefonia* y *wdbc* proporcionados en Moodle:

3. Estructura de clasificadores

Como se comentó en la introducción, los clasificadores se implementarán siguiendo los principios de diseño que se consideren más adecuados en función del ciclo de vida siguiente:

```
Crear Particiones (Validación Simple/Validación Cruzada)
For i=1 in range(Número-Particiones):
    Entrenar(datos train)
    Clasificar(datos de test)
    Calcular error de clasificación de la partición-i
Calcular error de clasificación total
```

Las dos operaciones específicas de cada clasificador son **entrenar** y **clasificar**. La primera es la que permite construir un modelo de aprendizaje utilizando los datos del conjunto de train, mientras que la segunda proporciona las predicciones sobre el conjunto de datos de test.

4. Estandarización de datos

Es muy habitual, como paso previo a la aplicación de algunos algoritmos de aprendizaje, tener que realizar una estandarización de los datos con el objetivo de



ajustar las escalas de los atributos numéricos y evitar así sesgos no deseados por la diferencia de escala. Se debe por tanto ampliar nuestra librería *MyScikitLearn* con esta funcionalidad implementando la siguiente operación en el lugar más apropiado, justificando la elección:

- `estandarizarDatos(media=True, std=True)`: estandariza cada uno de los atributos numéricos en la matriz de datos que se construyó en la práctica de introducción, utilizando la media del atributo si *media=True*, o cero si *media=False* y la desviación típica del atributo si *std=True*, o uno si *std=False*. Es decir, su comportamiento será el mismo al de la función correspondiente de la librería de Scikit learn que se comentará en el apartado 5.4.

5. Scikit Learn

5.1 Naive Bayes

Scikit Learn proporciona diferentes implementaciones para el algoritmo Naive Bayes (http://scikit-learn.org/stable/modules/naive_bayes.html). En esta práctica, consideraremos las siguientes funciones:

- **MultinomialNB**: es la versión para datos distribuidos multinomialmente. El valor a priori de cada clase se puede asumir uniforme (`fit_prior=False`) o se puede obtener a partir de los datos (`fit_prior=True`). Esta función también permite especificar un parámetro aditivo de suavizado `alpha`. `alpha=0` no realiza ningún tipo de suavizado y `alpha=1` implementa la corrección de Laplace. Este tipo de distribución se utiliza clásicamente en problemas de análisis de texto, como la clasificación de correo en spam/no spam, ya que la distribución multinomial no representa categorías, sino cuentas, por ejemplo, de palabras.
- **GaussianNB**: la verosimilitud de las variables se asume que sigue una distribución normal.
- **CategoricalNB**: es la versión Naive Bayes para datos categóricos. Asume que cada característica tiene su propia distribución categórica. Como **MultinomialNB**, permite indicar los argumentos `fit_prior` y `Alpha`.

5.2 K-NN

Scikit-learn implementa dos clasificadores basados en vecindad. Para los propósitos de esta práctica interesa *KNeighborsClassifier*. Un argumento de este método es **metric**, que permite indicar la distancia a utilizar.

5.3 Particionado

En cuanto al particionado de los datos, el submódulo **model_selection** de Scikit-



learn implementa diferentes estrategias (http://scikit-learn.org/stable/model_selection.html). En lo que se refiere a las estrategias de validación simple y validación cruzada a implementar en esta práctica, estas son algunas de las funciones más relevantes:

- **Validación simple:** La función `train_test_split` hace transparente la gestión de índices ya que devuelve directamente los conjuntos de entrenamiento y validación para un conjunto de datos dado. Por su parte, la instanciación de la clase `ShuffleSplit` proporciona los índices para dividir los datos en subconjuntos de training y test. Los parámetros de estas funciones/constructores permiten indicar la proporción de patrones a utilizar en test o el número de particiones a generar, entre otros.
- **Validación cruzada:** Las funciones `cross_val_score` y `cross_val_predict` devuelven directamente los errores por validación cruzada por cada *fold* y las predicciones por cada patrón, respectivamente. Por su parte, la instanciación de la clase `KFold` genera los índices de las particiones, igual que hace la Estrategia de Particionado implementada en la práctica de introducción. Entre otras posibilidades, los parámetros de estas funciones/constructores permiten indicar el número de *folds* a generar o imponer la permutación aleatoria de los datos antes de generar la partición.

5.4 Estandarización de datos

La librería Scikit Learn proporciona el método `StandardScaler` (<https://scikit-learn.org/dev/modules/generated/sklearn.preprocessing.StandardScaler.html>) para realizar la estandarización de los datos, es decir, la conversión de los atributos de acuerdo a una distribución de media cero y desviación estándar uno.

6. Actividades

La planificación temporal sugerida y las actividades a llevar a cabo son las siguientes:

- *1ª semana:* Implementar el algoritmo *Naive-Bayes* y aplicarlo al análisis de los conjuntos de datos *fuga_telefonia* y *wdbc* para obtener los porcentajes de error en clasificación en cada caso, tanto con validación cruzada como con validación simple. En todos los casos se deben hacer los cálculos sin aplicar y aplicando la *corrección de Laplace*. Tened en cuenta que los dos conjuntos incluyen atributos continuos, por lo que en ese caso supondremos que estos atributos siguen una distribución normal dada la clase. Para ello se debe calcular la media y la varianza de cada atributo continuo. Pueden ser de utilidad las funciones para la distribución normal del módulo `scipy.stats.norm`.
- *2ª semana:* Comparar los resultados obtenidos en nuestra implementación con los que proporciona la librería de aprendizaje automático Scikit Learn (http://scikit-learn.org/stable/modules/naive_bayes.html). Reproducir el diseño



experimental utilizado anteriormente para la validación simple y la validación cruzada. Evaluar y comentar las hipótesis obtenidas con el clasificador Naive-Bayes según los criterios que se definen en el apartado 7 destinado a la entrega.

- *3ª semana:* Implementar el algoritmo de *vecinos próximos* para realizar una tarea de clasificación de los conjuntos de datos *fuga_telefonía* y *wdbc*, probando con diferentes valores de vecindad ($K=5, 9$ y 13) y los dos tipos de particionado (simple y cruzada). En el uso del algoritmo de vecinos próximos es recomendable estandarizar los atributos. La estandarización es importante puesto que evita problemas de escala. Utilizad el método *estandarizarDatos* que se ha propuesto en el apartado 4. Como métrica de distancia nos vamos a limitar a la **Euclídea**.
- *4ª semana:* Analizar los conjuntos de datos *fuga_telefonía* y *wdbc* con la librería de Scikit Learn para vecinos próximos, utilizando los mismos valores de K y la distancia Euclídea, tanto en validación simple como en validación cruzada. Comparar con los resultados obtenidos con la implementación propia, tanto en datos estandarizados como sin estandarizar. Aquí se utilizará el método *StandardScale* de Scikit Learn para esta tarea.

7. Fecha de entrega y entregables

29 de Octubre de 2026. La entrega debe realizarse antes del comienzo de la clase de prácticas correspondiente. Se deberá entregar un fichero comprimido .zip con nombre **FAAP1_<grupo>_<pareja>.zip** (ejemplo FAAP1_1461_1.zip) y el siguiente contenido:

1. **Notebook (.ipynb).** El Notebook debe estructurarse para contener los siguientes apartados:

Apartado 1	Naive-Bayes propio • Tabla con los resultados de la ejecución para los conjuntos de datos analizados (<i>fuga_telefonía</i> y <i>wdbc</i>). Considerar los dos tipos de particionado. Los resultados se refieren a las tasas de error y deben mostrarse tanto con la corrección de Laplace como sin ella. Se debe incluir tanto el promedio de error como su desviación típica. Es importante mostrar todos los resultados agrupados en una tabla para facilitar su evaluación. • Breve análisis de los resultados anteriores. Discutir el efecto Laplace
Apartado 2	Naive-Bayes Scikit Learn • Tabla de resultados equivalente a la anterior, pero utilizando los métodos del paquete Scikit Learn: MultinomialNB, GaussianNB y CategoricalNB. • Análisis comparativo de los tres métodos de Scikit Learn. ¿Existe algún problema con alguno de estos métodos en



	alguno de los dos ficheros? En caso afirmativo, ¿en cuál? ¿por qué? ¿cómo podría resolverse?
Apartado 3	K-NN propio - Resultados en forma de tabla de la clasificación mediante vecinos próximos para los diferentes valores de vecindad en los conjuntos de datos propuestos. Obtener los resultados tanto para datos estandarizados como sin estandarizar, con el objetivo de justificar el rendimiento del algoritmo en base a estas características. - Análisis de resultados: impacto de K (5,9,13) y de la estandarización (SI/NO)
Apartado 4	K-NN Scikit-Learn - Tabla de resultados equivalente a la del apartado anterior para las ejecuciones realizadas con la librería <i>KNeighborsClassifier</i> en los mismos valores de K, datos estandarizados y no estandarizados. - Análisis de resultados: impacto de K (5,9,13) y de la estandarización (SI/NO)
Apartado 5	Conclusión Comparar y analizar a modo de resumen los resultados propios con los de Scikit Learn para los dos algoritmos de aprendizaje y ambos conjuntos. Utiliza una tabla para comparar los resultados

2. Notebook exportado como fichero html.

3. Código Python (**ficheros.py**) necesarios para la correcta ejecución del Notebook. Esto incluye todos los ficheros implementados hasta ahora para el preprocesamiento, el particionado y la clasificación, tanto de la implementación propia como de Scikit Learn.

8. Puntuación de cada apartado

La valoración de cada apartado será la siguiente:

- ✓ **Naive-Bayes propio:** 2,5 puntos.
- ✓ **Naive-Bayes con Scikit-Learn:** 1,5 puntos.
- ✓ **K-NN propio:** 2,5 puntos
- ✓ **K-NN con Scikit-Learn:** 1,5 puntos
- ✓ **Presentación y discusión de los resultados:** 2 puntos.